 # DSA Patterns

3 Powerful Patterns that enough to crack TCS nqt

Pattern 1

1. Two Sum – Arrays

Description: Find two indices such that their values sum to a target.

Java

```java
public int[] twoSum(int[] nums, int target)
{ Map<Integer, Integer> map = new
HashMap<>(); for (int i = 0; i < nums.length;
i++) {
int diff = target - nums[i];
if (map.containsKey(diff)) return new int[]{map.get(diff), i};
map.put(nums[i], i);
}
return new int[0];
}
```

C++

```cpp
vector<int> twoSum(vector<int>& nums, int target)
{ unordered_map<int, int> map;
```

```
for (int i = 0; i < nums.size(); i++)
{ int diff = target - nums[i];
if (map.count(diff)) return {map[diff], i};
map[nums[i]] = i;
}
return {};
}
```

Python

```python
def twoSum(nums, target):
hashmap = {}
for i, num in enumerate(nums):
diff = target - num
if diff in hashmap:
return [hashmap[diff], i]
hashmap[num] = i
return []
```

2. Kadane's Algorithm – Max Subarray Sum

Description: Find the maximum sum of a contiguous subarray.

Java

```java
```

```
public int maxSubArray(int[] nums)
{ int max = nums[0], curr =
nums[0];
for (int i = 1; i < nums.length; i++) {
curr = Math.max(nums[i], curr + nums[i]);
max = Math.max(max, curr);
}
return max;
}
```

C++

```cpp
int maxSubArray(vector<int>& nums) {
int maxSum = nums[0], curr = nums[0];
for (int i = 1; i < nums.size(); i++) {
curr = max(nums[i], curr + nums[i]);
maxSum = max(maxSum, curr);
}
return maxSum;
}
```

Python

```python
def maxSubArray(nums):
```

```
max_sum = curr = nums[0]
for num in nums[1:]:
curr = max(num, curr + num)
max_sum = max(max_sum, curr)
return max_sum
```

3. Valid Anagram – Strings

Description: Check if two strings are anagrams.

Java

```java
public boolean isAnagram(String s, String t)
{ if (s.length() != t.length()) return
false; int[] count = new int[26];
for (char c : s.toCharArray()) count[c - 'a']++;
for (char c : t.toCharArray()) if (--count[c - 'a'] < 0) return false;
return true;
```

}

C++

```cpp
bool isAnagram(string s, string t) {
if (s.size() != t.size()) return false;
vector<int> count(26, 0);
```

```
for (char c : s) count[c - 'a']++;
for (char c : t) if (--count[c - 'a'] < 0) return false;
return true;
}
```

Python

```python
def isAnagram(s, t):
return sorted(s) == sorted(t)
```

4. Longest Palindromic Substring – Strings

Description: Find the longest palindromic substring in a given string.

Java

```java
public String longestPalindrome(String s)
{ int start = 0, maxLen = 0;
for (int i = 0; i < s.length(); i++) {
int len1 = expand(s, i, i), len2 = expand(s, i, i + 1);
int len = Math.max(len1, len2);
if (len > maxLen)
{ maxLen = len;
start = i - (len - 1) / 2;
}
```

```
}
return s.substring(start, start + maxLen);
}
private int expand(String s, int l, int r) {
while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r))
{ l--; r++;
}
return r - l - 1;
}
```

C++:

```
cpp
string longestPalindrome(string s)
{ int start = 0, maxLen = 0;
for (int i = 0; i < s.size(); i++) {
int len1 = expand(s, i, i), len2 = expand(s, i, i + 1);

int len = max(len1, len2);
if (len > maxLen) {
maxLen = len;
start = i - (len - 1) / 2;
}
}
return s.substr(start, maxLen);
}
int expand(string s, int l, int r) {
while (l >= 0 && r < s.size() && s[l] == s[r]) { l--; r++; }
```

return r - l - 1;

}

Python

```python
def longestPalindrome(s):
start = max_len = 0
for i in range(len(s)):
for l, r in [(i, i), (i, i+1)]:
while l >= 0 and r < len(s) and s[l] == s[r]:
```

l -= 1

r += 1

if r - l - 1 > max_len:

start = l + 1

max_len = r - l - 1

return s[start:start + max_len]

5. Reverse Linked List – Linked List

Description: Reverse a singly linked list.

Java

```java
public ListNode reverseList(ListNode head)
{ ListNode prev = null;
while (head != null)
{ ListNode next =
```

head.next; head.next =

prev;

prev = head;

head = next;

}

return prev;

}

```

C++:

cpp

ListNode* reverseList(ListNode* head)

{ ListNode* prev = nullptr;

while (head) {

ListNode* next = head->next;

head->next = prev;

prev = head;

head = next;

}

return prev;

}

**### Python**

```python

def reverseList(head):

prev = None

while head:

nxt = head.next

```

head.next = prev

prev = head

head = nxt

return prev

6. Add Two Numbers – Linked List

Description: Add two numbers represented as linked lists.

Java

```java

public ListNode addTwoNumbers(ListNode l1, ListNode l2)

{ ListNode dummy = new ListNode(0), curr = dummy;

int carry = 0;

while (l1 != null || l2 != null || carry != 0)

{ int sum = (l1 != null ? l1.val : 0) +

(l2 != null ? l2.val : 0) + carry;

carry = sum / 10;

curr.next = new ListNode(sum % 10);

curr = curr.next;

if (l1 != null) l1 = l1.next;

if (l2 != null) l2 = l2.next;

}

return dummy.next;

}

```
```

C++:

cpp

ListNode* addTwoNumbers(ListNode* l1, ListNode* l2)

{ ListNode* dummy = new ListNode(0), *curr =

dummy; int carry = 0;

while (l1 || l2 || carry) {

int sum = (l1 ? l1->val : 0) +

(l2 ? l2->val : 0) + carry;

carry = sum / 10;

curr->next = new ListNode(sum % 10);

curr = curr->next;

if (l1) l1 = l1->next;

if (l2) l2 = l2->next;

}

return dummy->next;

}

Python

```python

def addTwoNumbers(l1, l2):

dummy = ListNode(0)

curr = dummy

carry = 0

while l1 or l2 or carry:

sum = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry

carry = sum // 10
```

```

curr.next = ListNode(sum % 10)

curr = curr.next

l1 = l1.next if l1 else None

l2 = l2.next if l2 else None

return dummy.next

### 7. Merge Intervals – Arrays

Description: Merge overlapping intervals.

### Java

```java

public int[][] merge(int[][] intervals) {

Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

List<int[]> result = new ArrayList<>();

for (int[] interval : intervals) {

if (result.isEmpty() || result.get(result.size() - 1)[1] <

interval[0]) {

result.add(interval);

} else {

```

result.get(result.size() - 1)[1] =

Math.max(result.get(result.size() - 1)[1], interval[1]);

}

}

return result.toArray(new int[result.size()][]);
```

}

C++

```cpp
vector<vector<int>> merge(vector<vector<int>>& intervals)
{ sort(intervals.begin(), intervals.end());
vector<vector<int>> result;
for (auto& interval : intervals) {
if (result.empty() || result.back()[1] < interval[0])
{ result.push_back(interval);
} else {
result.back()[1] = max(result.back()[1], interval[1]);
}
}
```

return result;
}

Python

```python
def merge(intervals):
```

intervals.sort(key=lambda x: x[0])
result = []
for interval in intervals:
if not result or result[-1][1] < interval[0]:

result.append(interval)

else:

result[-1][1] = max(result[-1][1], interval[1])

return result

8. Find Missing Number – Arrays

Description: Find the missing number in an array of 1 to N.

Java

```java
public int missingNumber(int[] nums)
{ int n = nums.length;
int sum = n * (n + 1) / 2;
int arrSum = 0;
for (int num : nums)
{ arrSum += num;
}
return sum - arrSum;
}
```

C++

```cpp
int missingNumber(vector<int>& nums)
{ int n = nums.size();
int sum = n * (n + 1) / 2;
int arrSum = accumulate(nums.begin(), nums.end(), 0);
```

return sum - arrSum;

}

```

### Python

```python

def missingNumber(nums):

n = len(nums)

return n * (n + 1) // 2 - sum(nums)

```

9. Word Search – Backtracking

Description: Find if a word exists in a 2D grid of characters.

Java

```java

public boolean exist(char[][] board, String word)

{ for (int i = 0; i < board.length; i++) {

for (int j = 0; j < board[0].length; j++) {

if (backtrack(board, word, i, j, 0)) return true;

}

}

return false;

}

private boolean backtrack(char[][] board, String word, int i, int j, int

index) {

```
if (index == word.length()) return true;
if (i < 0 || j < 0 || i >= board.length || j >= board[0].length ||
board[i][j] != word.charAt(index)) return false;
char temp = board[i][j];
board[i][j] = '#'; // mark as visited
```

```
boolean found = backtrack(board, word, i+1, j, index+1) ||
backtrack(board, word, i-1, j, index+1) ||
backtrack(board, word, i, j+1, index+1) ||
backtrack(board, word, i, j-1, index+1);
board[i][j] = temp; // restore
return found;
}
```

C++

```cpp
bool exist(vector<vector<char>>& board, string word)
{ for (int i = 0; i < board.size(); i++) {
for (int j = 0; j < board[0].size(); j++) {
if (backtrack(board, word, i, j, 0)) return true;
}
}
return false;
}
bool backtrack(vector<vector<char>>& board, string word, int i, int j, int
index) {
if (index == word.length()) return true;
```

```
if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size() ||
board[i][j] != word[index]) return false;
char temp = board[i][j];
board[i][j] = '#';
```

```
bool found = backtrack(board, word, i+1, j, index+1) ||
backtrack(board, word, i-1, j, index+1) ||

backtrack(board, word, i, j+1, index+1) ||
backtrack(board, word, i, j-1, index+1);
board[i][j] = temp;
return found;
}
```

Python

```python
def exist(board, word):
for i in range(len(board)):
for j in range(len(board[0])):
if backtrack(board, word, i, j, 0):
return True
return False
def backtrack(board, word, i, j, index):
if index == len(word):
return True
if i < 0 or j < 0 or i >= len(board) or j >= len(board[0]) or
```

board[i][j] != word[index]:

return False

temp = board[i][j]

board[i][j] = '#' # mark as visited

found = (backtrack(board, word, i + 1, j, index + 1) or

backtrack(board, word, i - 1, j, index + 1) or

backtrack(board, word, i, j + 1, index + 1) or

backtrack(board, word, i, j - 1, index + 1))

board[i][j] = temp # restore

return found

10. Subsets – Backtracking

Description: Generate all possible subsets of a given set of numbers.

Java

```java

public List<List<Integer>> subsets(int[] nums)

{ List<List<Integer>> result = new

ArrayList<>(); backtrack(nums, 0, new

ArrayList<>(), result); return result;

}

private void backtrack(int[] nums, int start, List<Integer> current,

List<List<Integer>> result) {

result.add(new ArrayList<>(current));

for (int i = start; i < nums.length; i++)

{ current.add(nums[i]);

backtrack(nums, i + 1, current, result);
```

current.remove(current.size() - 1);

```

}

}

### C++

```cpp

vector<vector<int>> subsets(vector<int>& nums)

{ vector<vector<int>> result;

backtrack(nums, 0, {}, result);

return result;

}

void backtrack(vector<int>& nums, int start, vector<int> current,

vector<vector<int>>& result) {

result.push_back(current);

for (int i = start; i < nums.size(); i++)

{ current.push_back(nums[i]);

backtrack(nums, i + 1, current, result);

current.pop_back();

}

}

```

Python

```python

def subsets(nums):

result = []

```

backtrack(nums, 0, [], result)

return result

def backtrack(nums, start, current, result):

result.append(list(current))

for i in range(start, len(nums)):

current.append(nums[i])

backtrack(nums, i + 1, current, result)

current.pop()

Pattern 2

1. Easy: Find the Duplicate Number

Description: Find the duplicate number in an array containing n + 1 integers where each integer is between 1 and n.

Java

```java

public int findDuplicate(int[] nums)

{ int slow = nums[0], fast = nums[0];

do {

```

slow = nums[slow];

fast = nums[nums[fast]];

} while (slow != fast);
```

fast = nums[0];

while (slow != fast)

{ slow =

nums[slow]; fast =

nums[fast];

}

return slow;

}

C++

```cpp

int findDuplicate(vector<int>& nums)

{ int slow = nums[0], fast = nums[0];

do {

slow = nums[slow];

fast = nums[nums[fast]];

} while (slow != fast);

fast = nums[0];

while (slow != fast)

{ slow =

nums[slow]; fast =

nums[fast];

}

return slow;

}

```

Python

```python
def findDuplicate(nums):
slow = fast = nums[0]
while True:
slow = nums[slow]
fast = nums[nums[fast]]
if slow == fast:
```

break
fast = nums[0]
while slow != fast:
slow = nums[slow]
fast = nums[fast]
return slow

2. Easy: Merge Sorted Array

Description: Merge two sorted arrays into one sorted array.

Java

```java
public void merge(int[] nums1, int m, int[] nums2, int n)
{ int i = m - 1, j = n - 1, k = m + n - 1;
```

while (i >= 0 && j >= 0) {

if (nums1[i] > nums2[j]) {

nums1[k--] = nums1[i--];

} else {

nums1[k--] = nums2[j--];

}

}

while (j >= 0) {

nums1[k--] = nums2[j--];

}

}

C++

```cpp

void merge(vector<int>& nums1, int m, vector<int>& nums2, int n)

{ int i = m - 1, j = n - 1, k = m + n - 1;

while (i >= 0 && j >= 0) {

if (nums1[i] > nums2[j]) {

nums1[k--] = nums1[i--];

} else {

nums1[k--] = nums2[j--];

}

}

while (j >= 0) {

nums1[k--] = nums2[j--];

}

}

```

Python

```python
def merge(nums1, m, nums2, n):
```

```
i, j, k = m - 1, n - 1, m + n - 1
while i >= 0 and j >= 0:
if nums1[i] > nums2[j]:
nums1[k] = nums1[i]
i -= 1
else:
nums1[k] = nums2[j]
j -= 1
k -= 1
while j >= 0:
nums1[k] = nums2[j]
j -= 1
k -= 1
```

3. Medium: Rotate Image

Description: Rotate a given n x n 2D matrix by 90 degrees (clockwise).

Java

```java
public void rotate(int[][] matrix) {
```

```
int n = matrix.length;
for (int i = 0; i < n / 2; i++) {
for (int j = i; j < n - i - 1; j++)
{ int temp = matrix[i][j];
matrix[i][j] = matrix[n - j - 1][i];
matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];
matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];
matrix[j][n - i - 1] = temp;
}
}
}
```

C++

```cpp
void rotate(vector<vector<int>>& matrix)
{ int n = matrix.size();
for (int i = 0; i < n / 2; i++) {
for (int j = i; j < n - i - 1; j++)
{ int temp = matrix[i][j];
matrix[i][j] = matrix[n - j - 1][i];
matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];
matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];
matrix[j][n - i - 1] = temp;
}
}
}
```

Python

```python
def rotate(matrix):
n = len(matrix)
for i in range(n // 2):
for j in range(i, n - i - 1):
temp = matrix[i][j]
matrix[i][j] = matrix[n - j - 1][i]
```

matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1]

matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1]

matrix[j][n - i - 1] = temp

4. Medium: Longest Substring Without Repeating Characters

Description: Find the length of the longest substring without repeating characters.

Java

```java
public int lengthOfLongestSubstring(String s)
{ Set<Character> set = new HashSet<>();
int left = 0, maxLength = 0;
for (int right = 0; right < s.length(); right++)
{ while (set.contains(s.charAt(right))) {
set.remove(s.charAt(left));
```

left++;

}

set.add(s.charAt(right));

maxLength = Math.max(maxLength, right - left + 1);

}

return maxLength;

}

C++:

cpp

int lengthOfLongestSubstring(string s)

{ unordered_set<char> set;

int left = 0, maxLength = 0;

for (int right = 0; right < s.size(); right++)

{ while (set.count(s[right])) {

set.erase(s[left]);

left++;

}

set.insert(s[right]);

maxLength = max(maxLength, right - left + 1);

}

return maxLength;

}

Python

```python

def lengthOfLongestSubstring(s):

char_set = set()

```

left, max_len = 0, 0

for right in range(len(s)):

while s[right] in char_set:

char_set.remove(s[left])

left += 1

char_set.add(s[right])

max_len = max(max_len, right - left + 1)

return max_len

5. Hard: N-Queens

Description: Solve the N-Queens puzzle by returning all distinct solutions.

Java

```java

public List<List<String>> solveNQueens(int n)

{ List<List<String>> result = new ArrayList<>();

solve(n, 0, new int[n], result);

return result;

}

private void solve(int n, int row, int[] cols, List<List<String>> result)

{ if (row == n) {

```

List<String> board = new ArrayList<>();

for (int i = 0; i < n; i++) {

char[] rowArr = new char[n];

Arrays.fill(rowArr, '.');

```
rowArr[cols[i]] = 'Q';
board.add(new String(rowArr));
}
result.add(board);
return;
}
for (int col = 0; col < n; col++)
{ if (isValid(row, col, cols)) {
cols[row] = col;
solve(n, row + 1, cols, result);
}
}
}
private boolean isValid(int row, int col, int[] cols)
{ for (int i = 0; i < row; i++) {
if (cols[i] == col || Math.abs(cols[i] - col) == row - i)
{ return false;
}
}
return true;
}
```

C++

```cpp
vector<vector<string>> solveNQueens(int n)
{ vector<vector<string>> result;
vector<int> cols(n);
solve(n, 0, cols, result);
```

```
return result;
}
void solve(int n, int row, vector<int>& cols, vector<vector<string>>&
result) {
if (row == n) {
vector<string> board(n, string(n, '.'));
for (int i = 0; i < n; i++) {
board[i][cols[i]] = 'Q';
}
result.push_back(board);
return;
}
for (int col = 0
```

6. Medium: Container With Most Water

Description: Given an array of heights, find two lines that together with the x-axis form a container that holds the most water.

Java

```java
public int maxArea(int[] height) {
int left = 0, right = height.length - 1;
int maxArea = 0;
while (left < right) {
int width = right - left;
```

```
int minHeight = Math.min(height[left], height[right]);
maxArea = Math.max(maxArea, width * minHeight);
if (height[left] < height[right])
{ left++;
} else {
right--;
}
}
return maxArea;
}
```

C++

```cpp
int maxArea(vector<int>& height) {
int left = 0, right = height.size() - 1;
int maxArea = 0;
while (left < right) {
int width = right - left;
int minHeight = min(height[left], height[right]);
maxArea = max(maxArea, width * minHeight);
if (height[left] < height[right])
{ left++;
} else {
right--;
}
}
```

return maxArea;

}

```

### Python

```python

def maxArea(height):

```

left, right = 0, len(height) - 1

max_area = 0

while left < right:

width = right - left

min_height = min(height[left], height[right])

max_area = max(max_area, width * min_height)

if height[left] < height[right]:

left += 1

else:

right -= 1

return max_area

7. Hard: Merge K Sorted Lists

Description: Merge k sorted linked lists into one sorted list.

Java

```java

```
public ListNode mergeKLists(ListNode[] lists) {
if (lists == null || lists.length == 0) return null;
```

```
PriorityQueue<ListNode> pq = new
PriorityQueue<>(Comparator.comparingInt(a -> a.val));
for (ListNode list : lists) {
if (list != null) pq.offer(list);
}
ListNode dummy = new ListNode(0);
ListNode current = dummy;
while (!pq.isEmpty())
{ current.next = pq.poll();
current = current.next;
if (current.next != null) pq.offer(current.next);
}
return dummy.next;
}
```

C++

```cpp
ListNode* mergeKLists(vector<ListNode*>& lists)
{ if (lists.empty()) return nullptr;
auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
for (auto list : lists) {
if (list) pq.push(list);
}
```

```
ListNode* dummy = new ListNode(0);
ListNode* curr = dummy;
while (!pq.empty()) {
curr->next = pq.top();
pq.pop();
curr = curr->next;
if (curr->next) pq.push(curr->next);
}
return dummy->next;
}
```

Python:

```
python
import heapq
def mergeKLists(lists):
heap = []
for l in lists:
if l:
heapq.heappush(heap, (l.val, l))

dummy = ListNode(0)
current = dummy
while heap:
val, node = heapq.heappop(heap)
current.next = node
current = current.next
if node.next:
```

heapq.heappush(heap, (node.next.val, node.next))

return dummy.next

8. Medium: Permutations

Description: Generate all permutations of a given list of numbers.

Java

```java
public List<List<Integer>> permute(int[] nums)
{ List<List<Integer>> result = new
ArrayList<>(); backtrack(nums, new
ArrayList<>(), result); return result;
}
private void backtrack(int[] nums, List<Integer> current,
List<List<Integer>> result) {
if (current.size() == nums.length)
{ result.add(new ArrayList<>(current));
return;
}
for (int num : nums) {
if (current.contains(num)) continue;
current.add(num);
backtrack(nums, current, result);
current.remove(current.size() - 1);
}
}
```

C++

```cpp
vector<vector<int>> permute(vector<int>& nums)
{ vector<vector<int>> result;
backtrack(nums, {}, result);
return result;
}
void backtrack(vector<int>& nums, vector<int> current, vector<vector<int>>&
result) {
if (current.size() == nums.size())
{ result.push_back(current);
return;
}
for (int num : nums) {
if (find(current.begin(), current.end(), num) != current.end())
continue;
```

```
current.push_back(num);
backtrack(nums, current, result);
current.pop_back();
}
}
```

Python

```python
def permute(nums):
```

result = []

```

backtrack(nums, [], result)

return result

def backtrack(nums, current, result):

if len(current) == len(nums):

result.append(list(current))

return

for num in nums:

if num in current: continue

current.append(num)

backtrack(nums, current, result)

current.pop()

**### 9. Hard: Word Search II**

Description: Find all words in a 2D board of letters using a dictionary.

**### Java**

```java

public List<String> findWords(char[][] board, String[] words)

{ Set<String> result = new HashSet<>();

TrieNode root = buildTrie(words);

boolean[][] visited = new boolean[board.length][board[0].length];

for (int i = 0; i < board.length; i++) {

for (int j = 0; j < board[0].length; j++)

{ dfs(board, i, j, root, visited, result);

}

```
}
return new ArrayList<>(result);
}
private void dfs(char[][] board, int i, int j, TrieNode node, boolean[][]
visited, Set<String> result) {
if (i < 0 || j < 0 || i >= board.length || j >= board[0].length ||
visited[i][j]) return;
char c = board[i][j];
if (!node.children.containsKey(c)) return;
visited[i][j] = true;
node = node.children.get(c);
if (node.word != null) result.add(node.word);
dfs(board, i + 1, j, node, visited, result);
dfs(board, i - 1, j, node, visited, result);
dfs(board, i, j + 1, node, visited, result);
```

```
dfs(board, i, j - 1, node, visited, result);
visited[i][j] = false;
}
private TrieNode buildTrie(String[] words)
{ TrieNode root = new TrieNode();
for (String word : words)
{ TrieNode node = root;
for (char c : word.toCharArray()) {
node = node.children.computeIfAbsent(c, k -> new TrieNode());
}
node.word = word;
```

}

return root;

}

class TrieNode {

Map<Character, TrieNode> children = new HashMap<>();

String word;

}

C++

```cpp

vector<string> findWords(vector<vector<char>>& board, vector<string>&

words) {

unordered_set<string> result;

TrieNode* root = buildTrie(words);

vector<vector<bool>> visited(board.size(),

vector<bool>(board[0].size(), false));

for (int i = 0; i < board.size(); i++) {

for (int j = 0; j < board[0].size(); j++)

{ dfs(board, i, j, root, visited, result);

}

}

return vector<string>(result.begin(), result.end());

}

void dfs(vector<vector<char>>& board, int i, int j, TrieNode* node,

vector<vector<bool>>& visited, unordered_set<string>& result) {

if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size() ||

visited[i][j]) return;

char c = board[i][j];
```

```
if (node->children.find(c) == node->children.end()) return;
visited[i][j] = true;
node = node->children[c];
if (!node->word.empty()) result.insert(node->word);
dfs(board, i + 1, j, node, visited, result);
dfs(board, i - 1, j, node, visited, result);
dfs(board, i, j + 1, node, visited, result);
dfs(board, i, j - 1, node, visited, result);
visited[i][j] = false;
}
TrieNode* buildTrie(vector<string>& words)
{ TrieNode* root = new TrieNode();
for (auto& word : words)
{ TrieNode* node =
root; for (char c :
word) {
```

```
if (node->children
```

10. Medium: Subsets

Description: Given a set of integers, return all possible subsets (the power set).

Java

```java
public List<List<Integer>> subsets(int[] nums)
{ List<List<Integer>> result = new
ArrayList<>(); backtrack(nums, 0, new
```

ArrayList<>(), result); return result;

}

private void backtrack(int[] nums, int start, List<Integer> current,

List<List<Integer>> result) {

result.add(new ArrayList<>(current));

for (int i = start; i < nums.length; i++)

{ current.add(nums[i]);

backtrack(nums, i + 1, current, result);

current.remove(current.size() - 1);

}

}

```

**### C++**

```cpp

vector<vector<int>> subsets(vector<int>& nums)

{ vector<vector<int>> result;

backtrack(nums, 0, {}, result);

return result;

}

void backtrack(vector<int>& nums, int start, vector<int> current,

vector<vector<int>>& result) {

result.push_back(current);

for (int i = start; i < nums.size(); i++)

{ current.push_back(nums[i]);

backtrack(nums, i + 1, current, result);

current.pop_back();

}

}

```

Python

```python
def subsets(nums):
result = []
```

backtrack(nums, 0, [], result)

return result

def backtrack(nums, start, current, result):

result.append(list(current))

for i in range(start, len(nums)):

current.append(nums[i])

backtrack(nums, i + 1, current, result)

current.pop()

1. Rotate Image

Description: You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

Java

```java
public void rotate(int[][] matrix)
```
```

```
{ int n = matrix.length;
for (int i = 0; i < n / 2; i++) {
for (int j = i; j < n - i - 1; j++)
{ int temp = matrix[i][j];
matrix[i][j] = matrix[n - j - 1][i];
matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];
matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];
matrix[j][n - i - 1] = temp;
}
}
}
```

C++

```cpp
void rotate(vector<vector<int>>& matrix)
{ int n = matrix.size();
for (int i = 0; i < n / 2; i++) {
for (int j = i; j < n - i - 1; j++)
{ int temp = matrix[i][j];
matrix[i][j] = matrix[n - j - 1][i];
matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1];
matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1];
matrix[j][n - i - 1] = temp;
}
}
}
```

```

### Python

```python
def rotate(matrix):
```

n = len(matrix)

for i in range(n // 2):

for j in range(i, n - i - 1):

temp = matrix[i][j]

matrix[i][j] = matrix[n - j - 1][i]

matrix[n - j - 1][i] = matrix[n - i - 1][n - j - 1]

matrix[n - i - 1][n - j - 1] = matrix[j][n - i - 1]

matrix[j][n - i - 1] = temp

2. Set Matrix Zeroes

Description: Given an m x n matrix, if an element is 0, set its entire row and column to 0.

Java

```java
public void setZeroes(int[][] matrix)
{ boolean rowZero = false, colZero = false;
for (int i = 0; i < matrix.length; i++) {
if (matrix[i][0] == 0)
{ rowZero = true;
break;
```
```

```
}
}
for (int j = 0; j < matrix[0].length; j++)
{ if (matrix[0][j] == 0) {
colZero = true;
break;
}
}
for (int i = 1; i < matrix.length; i++) {
for (int j = 1; j < matrix[0].length; j++)
{ if (matrix[i][j] == 0) {
matrix[i][0] = 0;
matrix[0][j] = 0;
}
}
}
for (int i = 1; i < matrix.length; i++) {
for (int j = 1; j < matrix[0].length; j++) {
if (matrix[i][0] == 0 || matrix[0][j] == 0)
{ matrix[i][j] = 0;
}
}
}
if (rowZero) {
for (int i = 0; i < matrix.length; i++)
{ matrix[i][0] = 0;
}
}
```

```
if (colZero) {
for (int j = 0; j < matrix[0].length; j++)
{ matrix[0][j] = 0;
}
}
}
```

C++

```cpp
void setZeroes(vector<vector<int>>& matrix)
{ bool rowZero = false, colZero = false;
for (int i = 0; i < matrix.size(); i++) {
if (matrix[i][0] == 0)
{ rowZero = true;
break;
}
}
for (int j = 0; j < matrix[0].size(); j++)
{ if (matrix[0][j] == 0) {
colZero = true;
break;
}
}
for (int i = 1; i < matrix.size(); i++) {
for (int j = 1; j < matrix[0].size(); j++)
{ if (matrix[i][j] == 0) {
```

```
matrix[i][0] = 0;
matrix[0][j] = 0;
}
}
}
for (int i = 1; i < matrix.size(); i++) {
for (int j = 1; j < matrix[0].size(); j++) {
if (matrix[i][0] == 0 || matrix[0][j] == 0)
{ matrix[i][j] = 0;
}
}
}
if (rowZero) {
for (int i = 0; i < matrix.size(); i++)
{ matrix[i][0] = 0;
}
}
if (colZero) {
for (int j = 0; j < matrix[0].size(); j++)
{ matrix[0][j] = 0;
}
}
}
```

Python

```python
```

```
def setZeroes(matrix):
row_zero = any(matrix[i][0] == 0 for i in range(len(matrix)))
col_zero = any(matrix[0][j] == 0 for j in range(len(matrix[0])))
for i in range(1, len(matrix)):
for j in range(1, len(matrix[0])):
if matrix[i][j] == 0:
matrix[i][0] = 0
matrix[0][j] = 0
for i in range(1, len(matrix)):
```

```
for j in range(1, len(matrix[0])):
if matrix[i][0] == 0 or matrix[0][j] == 0:
matrix[i][j] = 0
if row_zero:
for i in range(len(matrix)):
matrix[i][0] = 0
if col_zero:
for j in range(len(matrix[0])):
matrix[0][j] = 0
```

3. Word Search II

Description: Given a 2D board and a list of words, find all words in the board.

Java

```java
public List<String> findWords(char[][] board, String[] words)
{ Set<String> result = new HashSet<>();
```

```
TrieNode root = buildTrie(words);
boolean[][] visited = new boolean[board.length][board[0].length];
for (int i = 0; i < board.length; i++) {
for (int j = 0; j < board[0].length; j++)
{ dfs(board, i, j, root, visited, result);
}
}
return new ArrayList<>(result);
}
private void dfs(char[][] board, int i, int j, TrieNode node, boolean[][]
visited, Set<String> result) {
if (i < 0 || j < 0 || i >= board.length || j >= board[0].length ||
visited[i][j] || node.children[board[i][j] - 'a'] == null) return;
visited[i][j] = true;
node = node.children[board[i][j] - 'a'];
if (node.word != null) {
result.add(node.word);
}
dfs(board, i + 1, j, node, visited, result);
dfs(board, i - 1, j, node, visited, result);
dfs(board, i, j + 1, node, visited, result);
dfs(board, i, j - 1, node, visited, result);
visited[i][j] = false;
}
private TrieNode buildTrie(String[] words)
{ TrieNode root = new TrieNode();
for (String word : words)
{ TrieNode node = root;
```

for (char c : word.toCharArray()) {

```

if (node.children[c - 'a'] == null)

{ node.children[c - 'a'] = new TrieNode();

}

node = node.children[c - 'a'];

}

node.word = word;

}

return root;

}

class TrieNode {

TrieNode[] children = new TrieNode[26];

String word = null;

}

**### C++**

```cpp

class TrieNode

{ public:

TrieNode* children[26] = {};

string word;

};

vector<string> findWords(vector<vector<char>>& board, vector<string>&

words) {

set<string> result;

TrieNode* root = buildTrie(words);

```
vector<vector<bool>> visited(board.size(),
vector<bool>(board[0].size(), false));
for (int i = 0; i < board.size(); i++) {
for (int j = 0; j < board[0].size(); j++)
{ dfs(board, i, j, root, visited, result);
}
}
return vector<string>(result.begin(), result.end());
}
void dfs(vector<vector<char>>& board, int i, int j, TrieNode* node,
vector<vector<bool>>& visited, set<string>& result) {
if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size() ||
visited[i][j] || !node->children[board[i][j] - 'a']) return;
visited[i][j] = true;
node = node->children[board[i][j] - 'a'];
if (!node->word.empty()) {
result.insert(node->word);
}
dfs(board, i + 1, j, node, visited, result);
dfs(board, i - 1, j, node, visited, result);
dfs(board, i, j + 1, node, visited, result);
dfs(board, i, j - 1, node, visited, result);
visited[i][j] = false;
}
```

```
TrieNode* buildTrie(vector<string>& words)
{ TrieNode* root = new TrieNode();
```

for (string word : words)

{ TrieNode* node = root;

for (char c : word) {

if (!node->children[c - 'a']) {

node->children[c - 'a'] = new TrieNode();

}

node = node->children[c - 'a'];

}

node->word = word;

}

return root;

}

Python

```python

def findWords(board, words):

result = set()

trie = buildTrie(words)

visited = [[False] * len(board[0]) for _ in range(len(board))]

for i in range(len(board)):

for j in range(len(board[0])):

```

dfs(board, i, j, trie, visited, result)

return list(result)

def dfs(board, i, j, node, visited, result):

if i < 0 or j < 0 or i >= len(board) or j >= len(board[0]) or

visited[i][j]: return

char = board[i][j]

if char not in node: return

visited[i][j] = True

node = node[char]

if 'word' in node:

4. Merge Intervals

Description: Given a collection of intervals, merge all overlapping intervals.

Java

```java
public int[][] merge(int[][] intervals) {
if (intervals.length == 0) return new int[0][0];
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
List<int[]> merged = new ArrayList<>();
int[] current = intervals[0];
merged.add(current);
for (int[] interval : intervals)
{ if (interval[0] <= current[1])
{
current[1] = Math.max(current[1], interval[1]);
} else {
current = interval;
```

merged.add(current);

}

}

return merged.toArray(new int[merged.size()][]);

}

C++

```cpp
vector<vector<int>> merge(vector<vector<int>>& intervals)
{ if (intervals.empty()) return {};
sort(intervals.begin(), intervals.end());
vector<vector<int>> merged;
for (auto& interval : intervals) {
if (merged.empty() || merged.back()[1] < interval[0])
{ merged.push_back(interval);
} else {
merged.back()[1] = max(merged.back()[1], interval[1]);
}
}
return merged;
}
```

Python

```python
def merge(intervals):
if not intervals:
return []
```

intervals.sort(key=lambda x: x[0])

merged = [intervals[0]]

for interval in intervals[1:]:

if merged[-1][1] >= interval[0]:

merged[-1][1] = max(merged[-1][1], interval[1])

else:

merged.append(interval)

return merged

5. Unique Paths II

Description: A robot is located at the top-left corner of a m x n grid, it can only move down or right, and some cells are blocked. Find how many unique paths the robot can take.

Java

```java

public int uniquePathsWithObstacles(int[][] obstacleGrid)

{ int m = obstacleGrid.length;

int n = obstacleGrid[0].length;

int[] dp = new int[n];

dp[0] = obstacleGrid[0][0] == 1 ? 0 : 1;

for (int i = 0; i < m; i++) {

for (int j = 0; j < n; j++) {

if (obstacleGrid[i][j] == 1) dp[j] = 0;

```

else if (j > 0) dp[j] += dp[j - 1];

}

}

return dp[n - 1];

}

C++

```cpp
int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid)
{ int m = obstacleGrid.size();
int n = obstacleGrid[0].size();
vector<int> dp(n, 0);
dp[0] = obstacleGrid[0][0] == 1 ? 0 : 1;
for (int i = 0; i < m; i++) {
for (int j = 0; j < n; j++) {
if (obstacleGrid[i][j] == 1) dp[j] = 0;
else if (j > 0) dp[j] += dp[j - 1];
}
}
return dp[n - 1];
}
```

Python

```python
def uniquePathsWithObstacles(grid):
```

m, n = len(grid), len(grid[0])

dp = [0] * n

dp[0] = 1 if grid[0][0] == 0 else 0

for i in range(m):

for j in range(n):

if grid[i][j] == 1:

dp[j] = 0

elif j > 0:

dp[j] += dp[j - 1]

return dp[-1]

6. Best Time to Buy and Sell Stock

Description: You are given an array where prices[i] is the price of a given stock on day i.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Java

```java
public int maxProfit(int[] prices)
{ int minPrice = Integer.MAX_VALUE;
int maxProfit = 0;
for (int price : prices) {
```

minPrice = Math.min(minPrice, price);

maxProfit = Math.max(maxProfit, price - minPrice);

}

return maxProfit;

}

C++

```cpp
int maxProfit(vector<int>& prices)
{ int minPrice = INT_MAX;
int maxProfit = 0;
for (int price : prices) {
minPrice = min(minPrice, price);
maxProfit = max(maxProfit, price - minPrice);
}
return maxProfit;
}
```

Python

```python
def maxProfit(prices):
min_price = float('inf')
max_profit = 0
for price in prices:
min_price = min(min_price, price)
max_profit = max(max_profit, price - min_price)
return max_profit
```

7. Longest Palindromic Substring

Description: Given a string s, return the longest palindromic substring in s.

Java

```java
public String longestPalindrome(String s) {
if (s == null || s.length() == 0) return "";
int start = 0, maxLength = 1;
for (int i = 0; i < s.length(); i++) {
int len1 = expandAroundCenter(s, i, i);
int len2 = expandAroundCenter(s, i, i + 1);
int len = Math.max(len1, len2);
if (len > maxLength)
{ maxLength = len;
start = i - (maxLength - 1) / 2;
}
}
return s.substring(start, start + maxLength);
}
private int expandAroundCenter(String s, int left, int right) {
```

```
while (left >= 0 && right < s.length() && s.charAt(left) ==
s.charAt(right)) {
left--;
right++;
}
return right - left - 1;
}
```

C++:

cpp

string longestPalindrome(string s)

{ int start = 0, maxLength = 1;

for (int i = 0; i < s.size(); i++) {

int len1 = expandAroundCenter(s, i, i);

int len2 = expandAroundCenter(s, i, i + 1);

int len = max(len1, len2);

if (len > maxLength)

{ maxLength = len;

start = i - (maxLength - 1) / 2;

}

}

return s.substr(start, maxLength);

}

int expandAroundCenter(string s, int left, int right) {

while (left >= 0 && right < s.size() && s[left] == s[right])

{ left--;

right++;

}

return right - left - 1;

}

Python

```python

def longestPalindrome(s):

def expandAroundCenter(s, left, right):

while left >= 0 and right < len(s) and s[left] == s[right]:

```

left -= 1

right += 1

return right - left - 1

start, maxLength = 0, 1

for i in range(len(s)):

len1 = expandAroundCenter(s, i, i)

len2 = expandAroundCenter(s, i, i + 1)

length = max(len1, len2)

if length > maxLength:

maxLength = length

start = i - (maxLength - 1) // 2

return s[start:start + maxLength]

8. Jump Game II

Description: Given an array of non-negative integers nums, where each element represents your maximum jump length from that position, return the minimum number of jumps to reach the last index.

Java

```java

public int jump(int[] nums) {

int jumps = 0, currentEnd = 0, farthest = 0;

for (int i = 0; i < nums.length - 1; i++) {

farthest = Math.max(farthest, i + nums[i]);

if (i == currentEnd) {

jumps++;

currentEnd = farthest;
```

```
}
}
return jumps;
}
```

C++

```cpp
int jump(vector<int>& nums) {
int jumps = 0, currentEnd = 0, farthest = 0;
for (int i = 0; i < nums.size() - 1; i++) {
farthest = max(farthest, i + nums[i]);
if (i == currentEnd) {
jumps++;
currentEnd = farthest;
}
}
return jumps;
}
```

Python

```python
def jump(nums):
```

```
jumps, current_end, farthest = 0, 0, 0
for i in range(len(nums) - 1):
farthest = max(farthest, i + nums[i])
if i == current_end:
jumps += 1
current_end = farthest
return jumps
```

9. Decode Ways

Description: A message containing letters from A-Z can be encoded into numbers using the following mapping:

- 'A' -> "1", 'B' -> "2", ..., 'Z' -> "26".

Given a string s consisting of digits, determine the total number of ways to decode it.

Java

```java
public int numDecodings(String s) {
if (s == null || s.length() == 0) return 0;
int prev = 1, curr = s.charAt(0) == '0' ? 0 : 1;
for (int i = 1; i < s.length(); i++) {
int temp = curr;
if (s.charAt(i) == '0')
{ curr = 0;
}
if (s.charAt(i - 1) == '1' || (s.charAt(i - 1) == '2' &&
s.charAt(i) <= '6')) {
curr += prev;
```

```
}
prev = temp;
}
return curr;
}
```

C++

```cpp
int numDecodings(string s) {
if (s.empty() || s[0] == '0') return 0;
int prev = 1, curr = 1;
for (int i = 1; i < s.size(); i++)
{ int temp = curr;
if (s[i] == '0') curr = 0;
if (s[i - 1] == '1' || (s[i - 1] == '2' && s[i] <= '6')) curr +=
prev;
}
prev = temp;
return curr;
}
```

Python

```python
def numDecodings(s):
```

if not s or s[0] == '0': return 0

```

prev, curr = 1, 1

for i in range(1, len(s)):

temp = curr

if s[i] == '0': curr = 0

if s[i - 1] == '1' or (s[i - 1] == '2' and s[i] <= '6'):

curr += prev

prev = temp

return curr

**### 10. Word Break**

Description: Given a non-empty string s and a dictionary of words wordDict, determine if s can be segmented into a space-separated sequence of one or more dictionary words.

**### Java**

```java

public boolean wordBreak(String s, List<String> wordDict)

{ Set<String> wordSet = new HashSet<>(wordDict);

boolean[] dp = new boolean[s.length() + 1];

dp[0] = true;

for (int i = 1; i <= s.length(); i++)

{ for (int j = 0; j < i; j++) {

if (dp[j] && wordSet.contains(s.substring(j, i)))

{ dp[i] = true;

break;

}

```
}
}
return dp[s.length()];
}
```

C++

```cpp
bool wordBreak(string s, vector<string>& wordDict)
{ unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
vector<bool> dp(s.size() + 1, false);
dp[0] = true;
for (int i = 1; i <= s.size(); i++)
{ for (int j = 0; j < i; j++) {
if (dp[j] && wordSet.count(s.substr(j, i - j)))
{ dp[i] = true;
break;
}
}
}
return dp[s.size()];
}
```

Python

```python
```

```
def wordBreak(s, wordDict):
wordSet = set(wordDict)
dp = [False] * (len(s) + 1)
dp[0] = True
for i in range(1, len(s) + 1):
for j in range(i):
if dp[j] and s[j:i] in wordSet:
dp[i] = True
```

break

return dp[len(s)]